

Algorytmy i struktury danych

Zajęcia laboratoryjne – 14 maja

Kopce

Kopiec (*heap*) to *prawie pełne drzewo binarne*.

1. Drzewo binarne to drzewo, w którym każdy węzeł ma dokładnie dwoje dzieci.
2. W kopcu, wszystkie poziomy drzewa są wypełnione od lewej do prawej.
3. Ostatni poziom może być niepełny, ale nadal wypełniony od lewej do prawej.

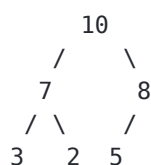
Te warunki pozwalają przechowywać kopiec w zwyczajnej tablicy.

Mamy dwa rodzaje kopców:

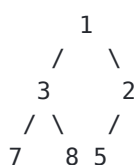
- **max-heap**: każdy węzeł przechowuje wartość liczbową **większą** od wartości, które przechowują jego dzieci
- **min-heap**: każdy węzeł przechowuje wartość liczbową **mniejszą** od wartości, które przechowują jego dzieci

Stąd wynika, że korzeń przechowuje największą/najmniejszą wartość w całym drzewie.

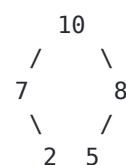
kopiec typu max



kopiec typu min



to nie jest kopiec



Jak przechowywać kopiec w tablicy? Stosujemy następujące zasady:

1. Wartość w tablicy pod indeksem $i=0$ zajmuje korzeń.
2. Jeżeli węzeł zajmuje pozycję pod indeksem i , to wówczas:
 - jego lewe dziecko umieszczamy pod indeksem $j = 2*i + 1$
 - jego prawe dziecko umieszczamy pod indeksem $j = 2*i + 2$
 - jego rodzic musi znajdować się pod indeksem $j = (i-1) // 2$

[Na Wikipedii](#) można znaleźć pomocny diagram obrazujący sytuację.

pop i push dla kopca typu min

push:

- Dodajemy element na koniec tablicy:**
 - Kopiec trzymamy w tablicy A.
 - Nowy element x dopisujemy na sam koniec:
`A.append(x)`.
 - W drzewie odpowiada to wstawieniu x w **pierwsze wolne miejsce** na najniższym poziomie (od lewej).
- Sprawdzamy, czy własność kopca jest spełniona:**
 - W min-heap: $A[\text{parent}(i)] \leq A[i]$ dla każdego wierzchołka i.
 - Niech i będzie indeksem nowego elementu: $i := \text{len}(A) - 1$.
 - Niech $p := (i - 1) // 2$ będzie indeksem jego rodzica.
- Podciągamy element w górę (sift-up):**
 - Jeśli $A[p] \leq A[i]$, to kończymy – kopiec jest poprawny.
 - Jeśli $A[p] > A[i]$, to własność kopca jest złamana:
 - zamieniamy element z rodzicem: $A[p] \leftrightarrow A[i]$,
 - przechodzimy na pozycję rodzica: $i := p$,
 - powtarzamy od 2.
- Kończymy, gdy:**
 - dojdziemy do korzenia ($i == 0$) albo
 - rodzic nie jest większy od dziecka ($A[p] \leq A[i]$).

pop:

- Zdejmujemy korzeń:**
 - W min-heapie najmniejsza wartość jest zawsze w korzeniu: $A[0]$.
 - Zapamiętujemy ją: $\text{top} := A[0]$.
- Wstawiamy ostatni element na górę:**
 - Zdejmujemy ostatni element tablicy: $\text{last} := A.\text{pop}()$.
 - Jeśli po tym tablica jest pusta, to zwracamy top i kończymy.
 - W przeciwnym razie ustawiamy: $A[0] := \text{last}$.
- Spychamy element w dół (sift-down):**
 - Niech $i := 0$ będzie indeksem elementu, który właśnie trafił na górę.
 - Liczymy dzieci:
 - $l := 2 * i + 1$ (lewe dziecko),
 - $r := 2 * i + 2$ (prawe dziecko).
 - Wybieramy mniejsze z dzieci (bo min-heap):
 - jeśli $A[l] < A[r]$, to wybieramy l,
 - w przeciwnym razie wybieramy r.
- Naprawiamy własność kopca:**
 - Jeśli $A[i] \leq A[\text{child}]$, to kończymy – kopiec jest poprawny.
 - Jeśli $A[i] > A[\text{child}]$, to własność kopca jest złamana:
 - zamieniamy $A[i] \leftrightarrow A[\text{child}]$,
 - schodzimy w dół: $i := \text{child}$,
 - Powtarzamy od 3.
- Zwracamy wynik:**
 - Po zakończeniu sift-down tablica A znów reprezentuje poprawny kopiec.
 - Zwracamy top – najmniejszy element, który był w korzeniu.

Kolejka priorytetowa

Kolejka priorytetowa to struktura danych, w której każdy element ma przypisany *priorytet*:

- W zwykłej kolejce elementy zdejmujemy w kolejności dodawania – *FIFO*.
 - W kolejce priorytetowej zdejmujemy element o najwyższym priorytecie (albo o najniższym, zależnie od konwencji, którą stosujemy).
-

Przykład. Chcemy skonstruować listę zadań do wykonania. Każde zadanie ma priorytet:

- odpisać na maila: 3
- zrobić zakupy: 2
- nauka do kolokwium: 5

Następne zadanie do wykonania to zadanie o najwyższym priorytecie. Skoro $5 > 3 > 2$, to pierwsze zadanie to nauka do kolokwium, drugie to odpisanie na maila, trzecie to zakupy. Po wykonaniu każdego zadania zdejmujemy je z kolejki.

Typowe operacje, których oczekujemy od implementacji kolejki priorytetowej:

- `push(x, p)` – dodaje element x o priorytecie p
- `pop()` – zdejmuje element o najwyższym / najniższym priorytecie
- `top()` – zwraca element z najwyższym / najniższym priorytetem bez usuwania
- `empty()` – sprawdza, czy kolejka jest pusta

Jeżeli kolejka priorytetowa jest zaimplementowana przy pomocy kopca, to:

- `push` działa w czasie $O(\log n)$
 - `pop` działa w czasie $O(\log n)$
 - `top` działa w czasie $O(1)$
-

W praktyce kolejki priorytetowe wykorzystujemy wtedy, gdy chcemy wielokrotnie dodawać nowe elementy i szybko wybierać element najlepszy według ustalonego priorytetu. Kolejki priorytetowe są zatem użyteczne np. w planowaniu zadań (jak w przykładzie) i przy konstruowaniu algorytmów wyszukiwania najkrótszej ścieżki w grafach.